AI Final Project

English Cursive Writing Recognition

Team 47　　C佳佳是一種可以慢嫚寫的程式語言

112550151 周佳瑩
112559198 簡嫚萱
111704035 蕭佳蘋

Table of Contents

01

Introduction

Motivation

- Once upon a time, one of our teammates came across some cursive English handwriting. She was shocked and said, "I can't recognize a single word!"
- It is difficult for people to recognize words in cursive or in inconsistent styles
- If even humans struggle, can machines do better?
- It is reported that many Americans struggle to read cursive handwriting nowadays
- Can you read cursive? The National Archives needs volunteers with your 'superpower'
- Gen Z Never Learned to Read Cursive

02

Method

Main Approach

Data Preprocessing

- **Convert all images to Grayscale**
  - transforms.Grayscale() converts them to single-channel
  - Reduces complexity and emphasizes shape/stroke features
- **Resize to 224x224**
  - Ensures all images have the same dimensions
  - Enables efficient batch training and to match input size of CNN
- **Convert to Tensor**
  - transforms.ToTensor() converts images to PyTorch tensors
  - Normalizes pixel values to the [0, 1] range

CNN – Feature Extraction

- **Convolutional Block Design**
  - Built 4 convolutional blocks (Conv → BatchNorm → ReLU → MaxPool)
  - Each block uses a 5×5 kernel with padding=1 to preserve spatial dimensions
  - 2×2 Max Pooling applied after each block to reduce feature map size

- **Dynamic Flatten Size Calculation**
  - Use dummy input to automatically compute the flatten input size
  - Avoids manual errors and ensures flexibility in architecture changes

CNN – Feature Extraction

- **Fully Connected Layers**
  - First layer: Linear(Flattened → 128) with Dropout(0.3) to reduce overfitting
  - Second layer: Linear(128 → 62) to classify 62 character classes
  - Output layer uses LogSoftmax to generate class probabilities

- **Additional Feature: Extract feature function**
  - Allows separation of feature extraction and sequence modeling components
  - Extracted features can be reused or fine-tuned in different contexts

BiLSTM

- **Why use BiLSTM in Our Model**
  - BiLSTM is a type of RNN that learns from both past and future context in a sequence.
  - Characters in handwriting depend not only on their own shape, but also on their neighbors.
  - BiLSTM helps the model understand full word-level structure, even with ambiguous or connected letters.

- **Our Implementation**
  - 2 BiLSTM layers, each with 256 hidden units
  - We apply Xavier initialization to improve training stability

CTC

- **Why we used CTC in Our Model**
  - In handwriting images, we don’t know exactly which part of the image corresponds to which character.
  - CTC solves this by allowing the model to learn alignments automatically.
- **Our Implementation**
  - Predicts a sequence of character probabilities over time steps
  - Allows “blank” tokens and repeated characters
  - Decodes to final output by removing blanks and merging repeats

For example : —ttt__iiimm__ee_ —> "time"

Beam Search with LM

- **Each candidate path is stored in a BeamEntry**
  - Used to track partial decoding hypotheses
  - Each entry holds the current prefix (sequence of predicted characters) and scores needed for beam ranking
- **Language model score (lm_score) is accumulated**
  - At each time step, the model evaluates the change in language model score when a new character is appended
- **Total score combines CTC and LM using alpha and beta**
  - Use log(probability) + α × LM_score + β × sequence_length
  - α (alpha) controls how much the language model influences the score.
  - β (beta) is a length bonus to prevent the model from favoring shorter sequences.

03

Results

Challenges

- **The accuracy of BiLSTM + CTC with decoding is too low on its own**
  - We combined it with CNN to see if the accuracy would improve
- **Accuracy for sentences is too low**
  - Due to time constraints, we ended up training CNN+BiLSTM+CTC+LM on words instead of sentences
  - Unable to access the full dataset for IAM Words, downloaded the subset on Kaggle instead
- **Dataset for CNN too small and lacks scarce characters like capital letters or v, x, z**
  - Combined two datasets to train the CNN, accuracy increased by at least 10%

Result

- **Developed a full English cursive character recognition with a 70.56% accuracy using 4-layered CNN**
  - Trained using English cursive characters

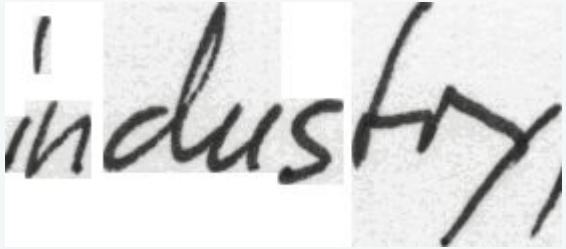
- **Developed English handwritten words recognition with a 67.10% accuracy using CNN + BiLSTM + CTC + Beam Search + LM**
  - CNN trained using English cursive characters
  - CNN + BiLSTM + CTC + Beam Search + LM trained using English handwritten words

Result

```
2025/06/02 22:59:40 [INFO]
[GT          ] industry
[Beam+Greedy] industry
2025/06/02 22:59:40 [INFO]
[GT          ] degre
[Beam+Greedy] dlegnee
2025/06/02 22:59:40 [INFO]
[GT          ] part
[Beam+Greedy] part
2025/06/02 22:59:40 [INFO]
[GT          ] are
[Beam+Greedy] are
2025/06/02 22:59:40 [INFO]
[GT          ] parties
[Beam+Greedy] pauties
2025/06/02 22:59:40 [INFO]
[GT          ] of
[Beam+Greedy] of
```

```
2025/06/03 01:52:07 [INFO]
[GT      ] industry
[Beam+LM] industry
2025/06/03 01:52:08 [INFO]
[GT      ] degre
[Beam+LM] dlegec
2025/06/03 01:52:08 [INFO]
[GT      ] part
[Beam+LM] part
2025/06/03 01:52:08 [INFO]
[GT      ] are
[Beam+LM] are
2025/06/03 01:52:08 [INFO]
[GT      ] parties
[Beam+LM] posties
2025/06/03 01:52:08 [INFO]
[GT      ] of
[Beam+LM] ofyf
```

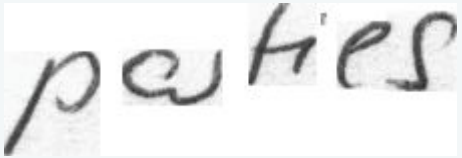
Result



Original data: industry
Prediction: industry

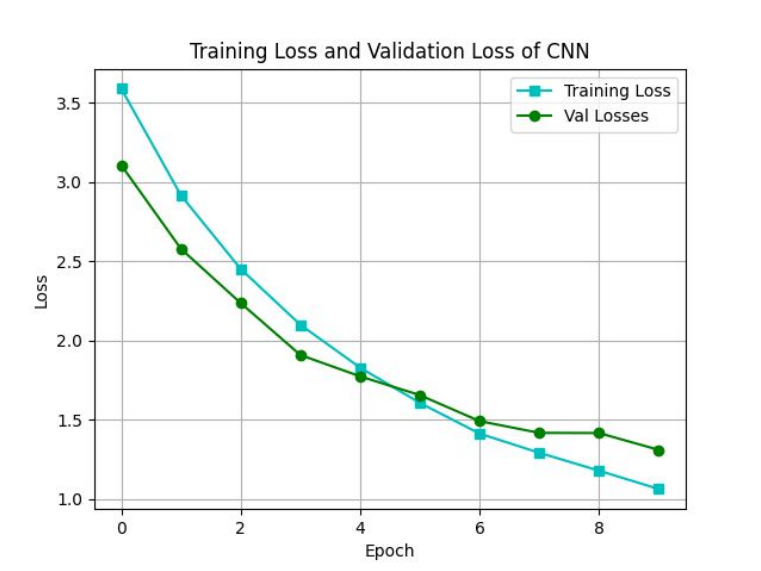
Original data: degree
Prediction: dlegnee, dlegec

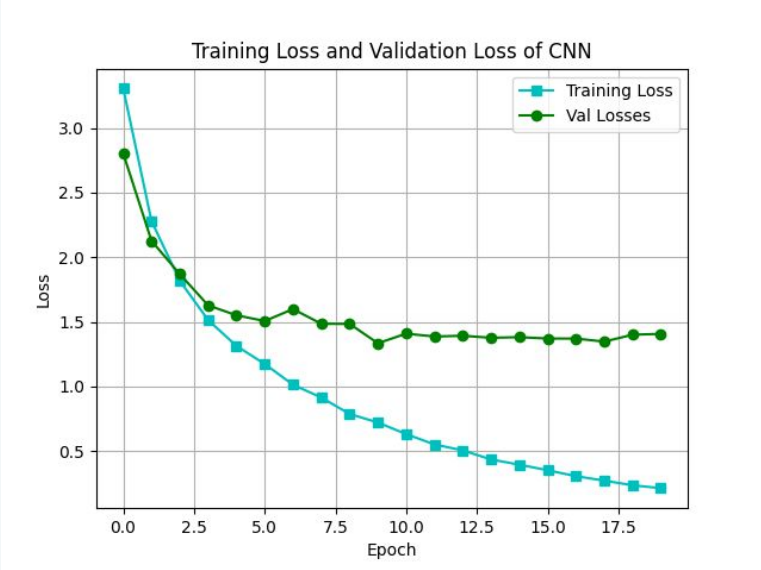
Original data: part
Prediction: part



Original data: parties
Prediction: pauties/ posties

Analyze Different CNN configurations



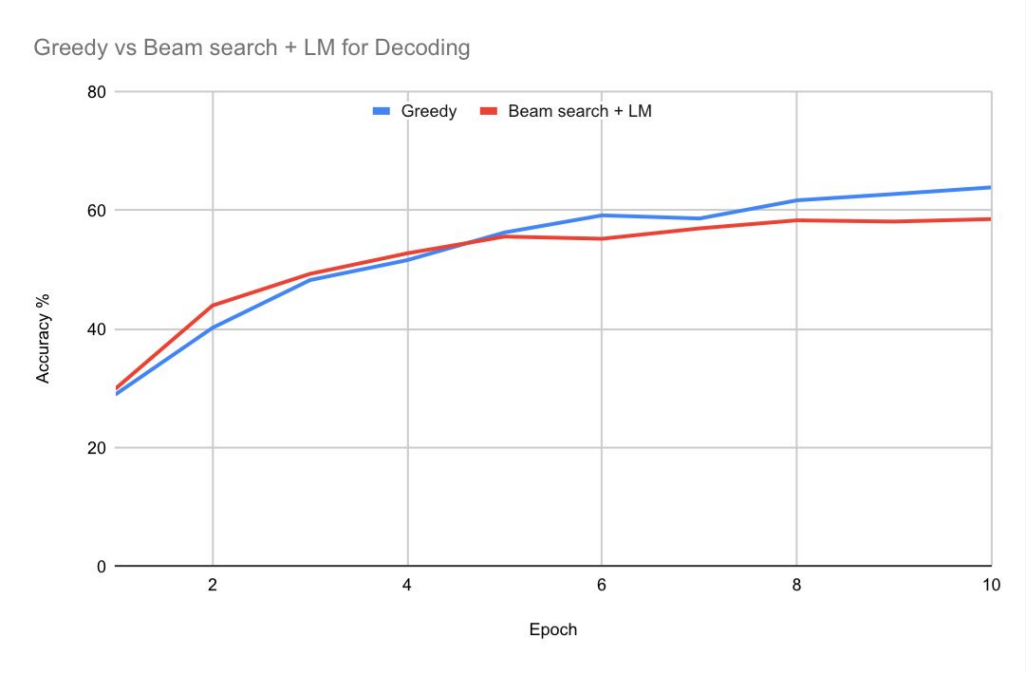


Highest CNN Accuracy (70.56%)

CNN (65.61%) for Highest CNN + BiLSTM + CTC + Beam Search + LM Accuracy

Analyze Greedy vs. LM for decoding

Highest accuracy of CNN + BiLSTM + CTC + Greedy: 63.90%
Highest accuracy of CNN + BiLSTM + CTC + Beam Search + LM: 58.55%



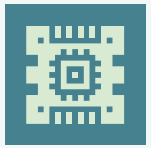
Originality Compared to previous work

Add Language Model (KenLM)

Improves decoding with sentence-level context
CTC Beam Search with LM → better predictions

Modular CNN Backbone

Design extract_feature() to reuse in hybrid models
Easier to extend



Dynamic Flatten Size Calculation

Avoids manual errors
Supports flexible architecture changes

Stronger Regularization

Use BatchNorm, Dropout(0.3), Max Pooling
Better generalization on small datasets

Future Aspirations

Doctor's Handwritten Prescription BD dataset

We tried training the CNN + BiLSTM + CTC+ Beam Search + LM on this dataset, but since the writing was messy and the characters weren't used to train the CNN, the results were disappointing. We could try segmenting the doctor's handwriting dataset to train the CNN.

Train CNN + BiLSTM + CTC + LM on sentences

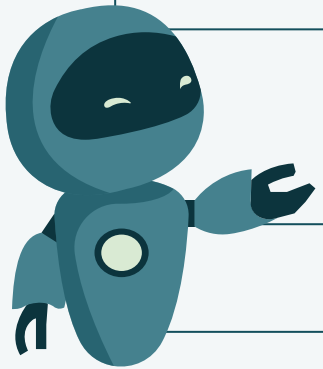
We tried training CNN + BiLSTM + CTC+ Beam Search + LM on sentences, but we might've skipped too far along and the model wasn't able to learn as well. We could try training the BiLSTM + CTC on words and training the a CNN + BiLSTM + CTC on words, then the full CNN + 2BiLSTM + CTC + LM on sentences.

04

Contributions

Member Contributions

簡嫚萱	周佳瑩	蕭佳蘋
Implement image processing, decode (greedy, beam search) and LM	Implement CNN using Lab 3 as a template and data loaders	Implement BiLSTM + CTC and data loaders
Train CNN, BiLSTM + greedy decoding Find datasets, make slides, record presentation video		
Everyone contributed equally much :D		



References

1. Cursive
    a. CHoiCe Dataset
    b. Cursive Handwriting Preprocessed by Jessica Mironyuk

2. Normal Handwriting
    a. IAM Handwriting Dataset
    b. IAM Word Dataset

3. Codes Used
    a. HandWritingRecognition by Arya Sarkar

Thanks!